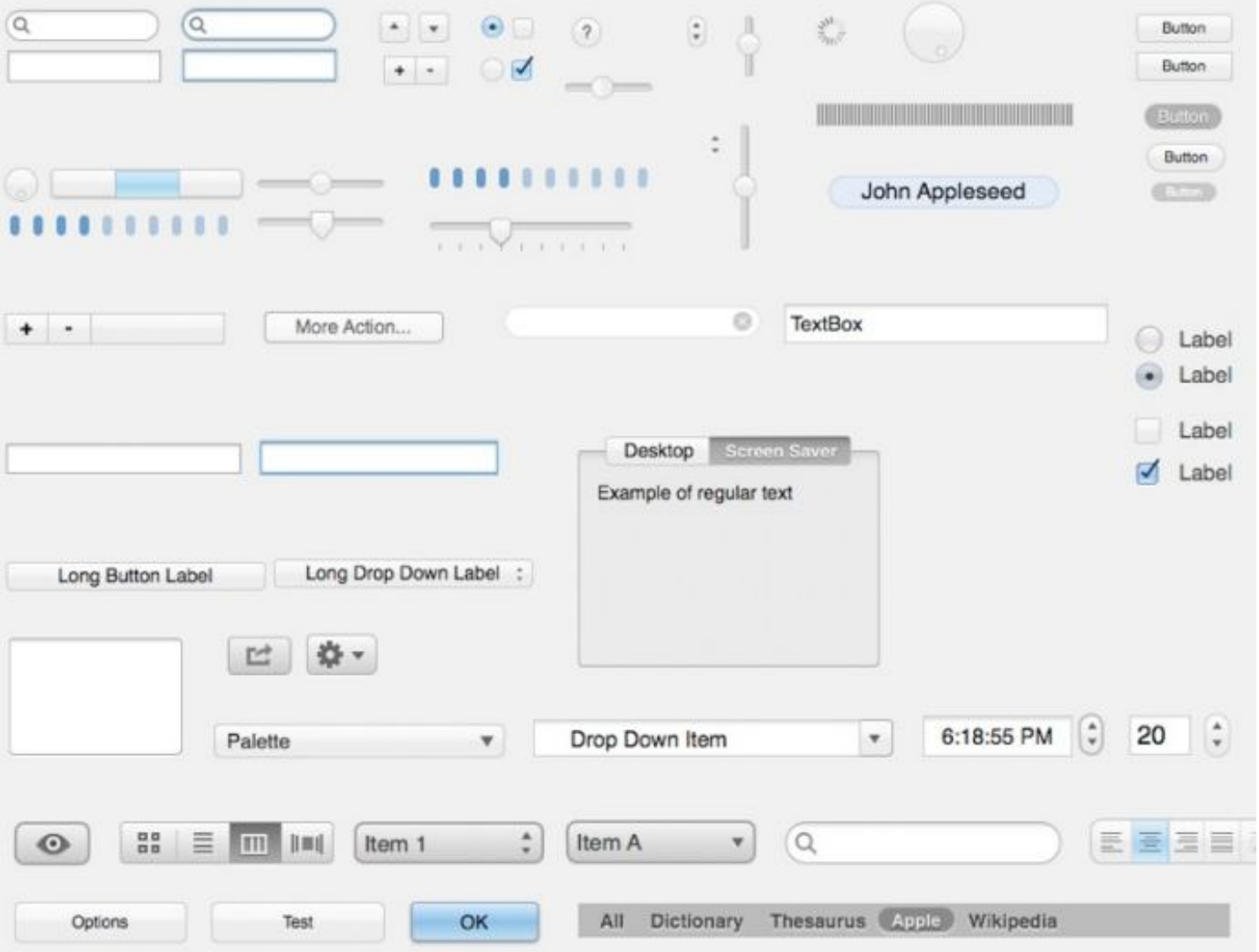
Elements of the WIMP interface

Besides windows, icons, menus, pointers
.. buttons, toolbars, palettes, dialog boxes

Different
toolkits
offer the same
set of widgets
(whose aspect
and behaviour
can differ)



Windows

Areas of the screen that **behave as if they were independent**

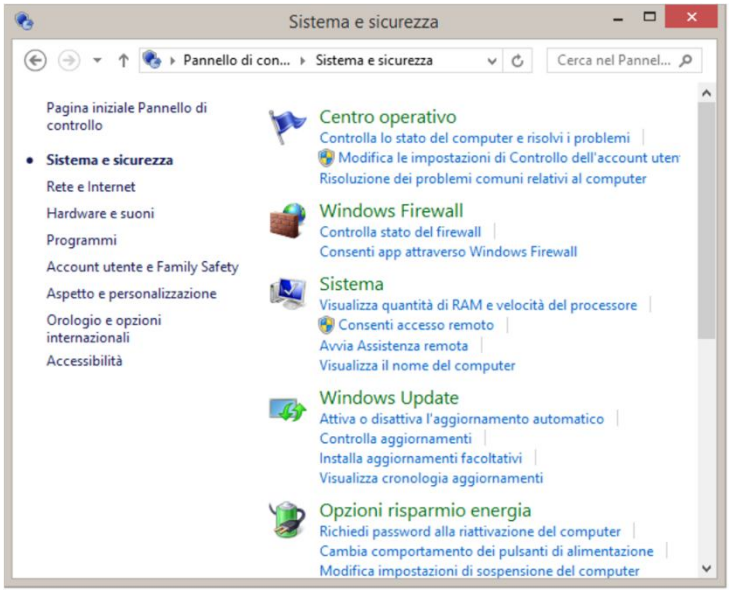
- can contain text or graphics
- can be moved or resized
- can overlap and obscure each other,
  - or can be laid out next to one another (tiled)

scrollbars

- allow the user to move the contents of the window up and down or from side to side

title bars

- describe the name of the window



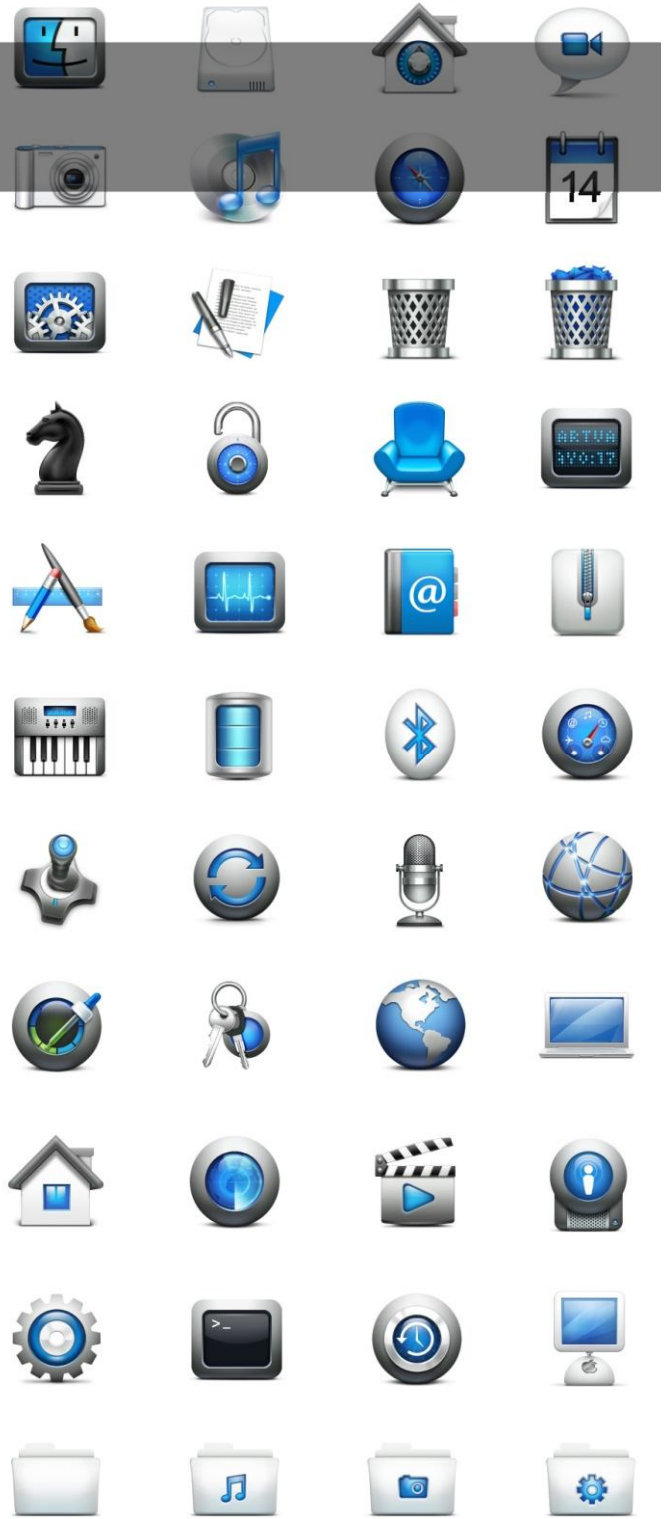
Icons

small picture or image

represents some object in the interface
- often a window or action

windows can be closed down (iconised)
- many accessible windows

icons can be many and various
- highly stylized
- realistic representations.



Icons advantages

With respect to textual representations

- allow a **faster and more precise recognition**
- require **less space**
- work (not always) with **different languages** and cultures

.. but they need to be properly designed in order not to confuse the user

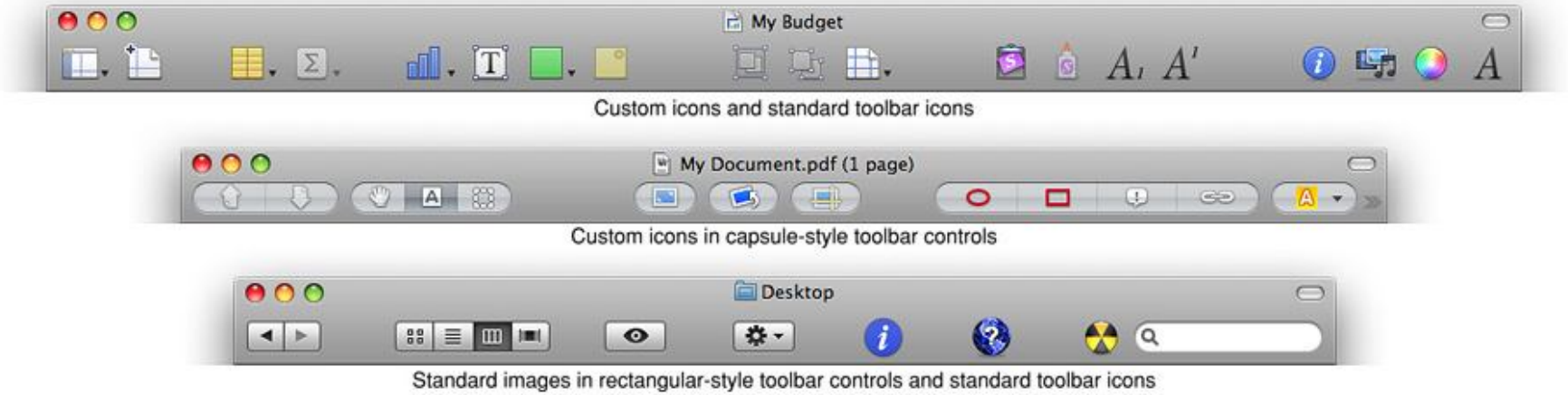
Use icons when..

- **quick and accurate recognition** of a message is necessary (e.g., warnings).
- displaying **visual or spatial concepts**
- the user (e.g. a driver) will be performing a **visual search of alternatives**.
- *continues…*

Use icons when..

- one **already exists** and has a generally **accepted meaning.**

- the **amount of space** on the display **is limited** and presenting the information textually will take up more space than is available.



Custom icons and standard toolbar icons

Custom icons in capsule-style toolbar controls

Standard images in rectangular-style toolbar controls and standard toolbar icons

Types of visual icons

- **Image-related**: highly pictorial representations of the object or act they represent (directly comprehended, use wherever possible)

- **Concept-related**: are based on an example or property of a real object or action (use if the user can be expected to comprehend the context in which the icon is presented)

- **Arbitrary**: become meaningful only through convention and education (can be difficult to recognize, hard to learn, and hard to remember). They should only be used if both context and special knowledge are present.

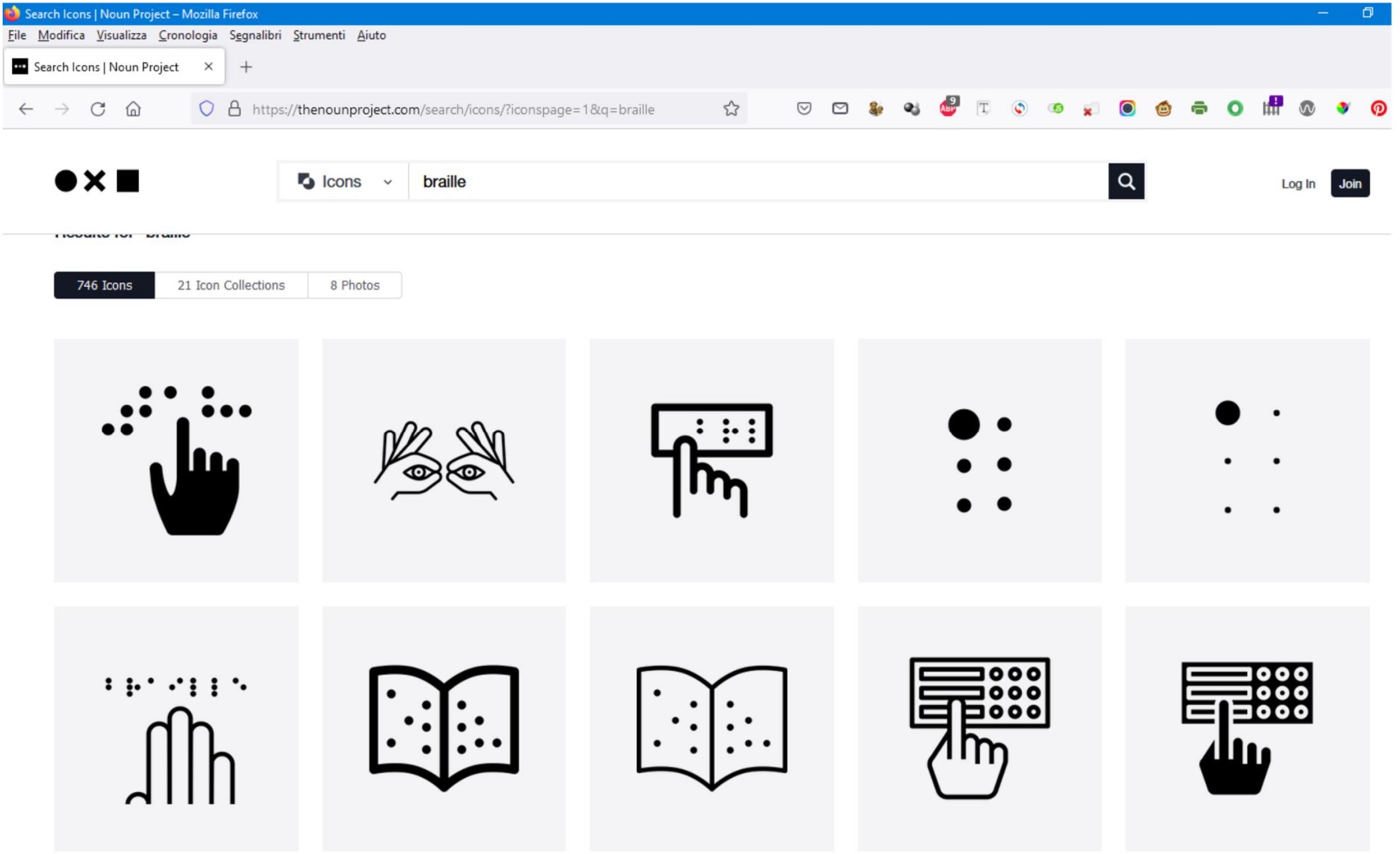
Tips for designing icons 1/2 Apple Human Interface Guidelines

- For great-looking icons, have a **professional graphic designer** create them.
- **Perspective and shadows** are the most important components of making good icons. Use a single light source with the light coming from above the icon.
- Use universal imagery that people will **easily recognize**. Avoid focusing on a secondary aspect of an element. For example, for a mail icon, a rural mailbox would be less recognizable than a postage stamp.
- Strive for simplicity. Try to use a **single object** that captures the icon's action or represents the control. Start with a **basic shape**.

Tips for designing icons 2/2 Apple Human Interface Guidelines

- Use **color judiciously** to help the icon tell its story; don't add color just to make the icon more colorful. Smooth gradients typically work better than sharp delineations of color.
- **Don't use replicas** of Apple hardware products in your icons. These symbols are copyrighted, and hardware designs change frequently.
- **Don't reuse** Mac OS X **system icons** in your interface; it can be confusing to users to see the same icon used to mean slightly different things in multiple locations.

thenounproject.com



Good vs Bad Icons

A **good icon** is clear, simple, and immediate.
A **bad** icon is complex, unclear, and slow to understand.

Good icons support recognition.
Bad icons require interpretation.

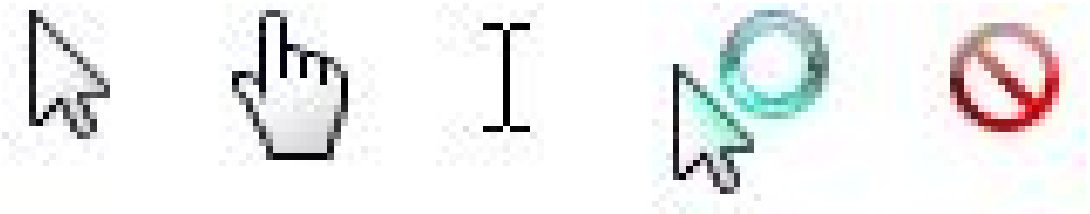
And in interface design,
we always prefer recognition over thinking.

Pointers

Important component
(WIMP style relies on pointing and selecting things)

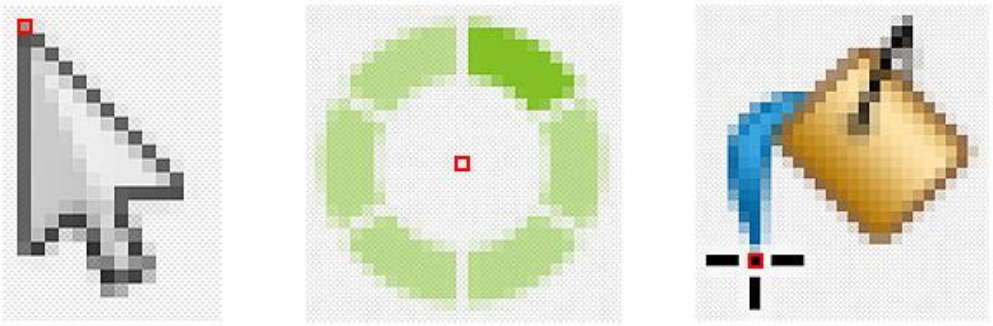
Use mouse, trackpad, joystick, trackball, cursor keys or keyboard shortcuts

Wide variety of graphical images to
- **distinguish** different **operating modes**
- inform about the **status of the system**
- indicate **sensible areas** (e.g. links)

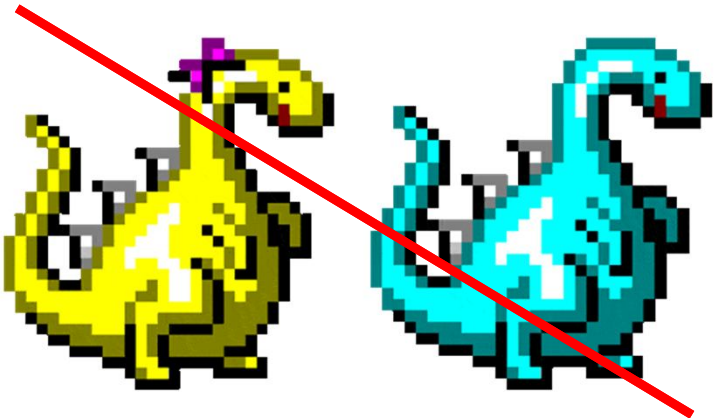


Pointers

The **active zone** (usually the tip of the arrow) must be **clearly identified**

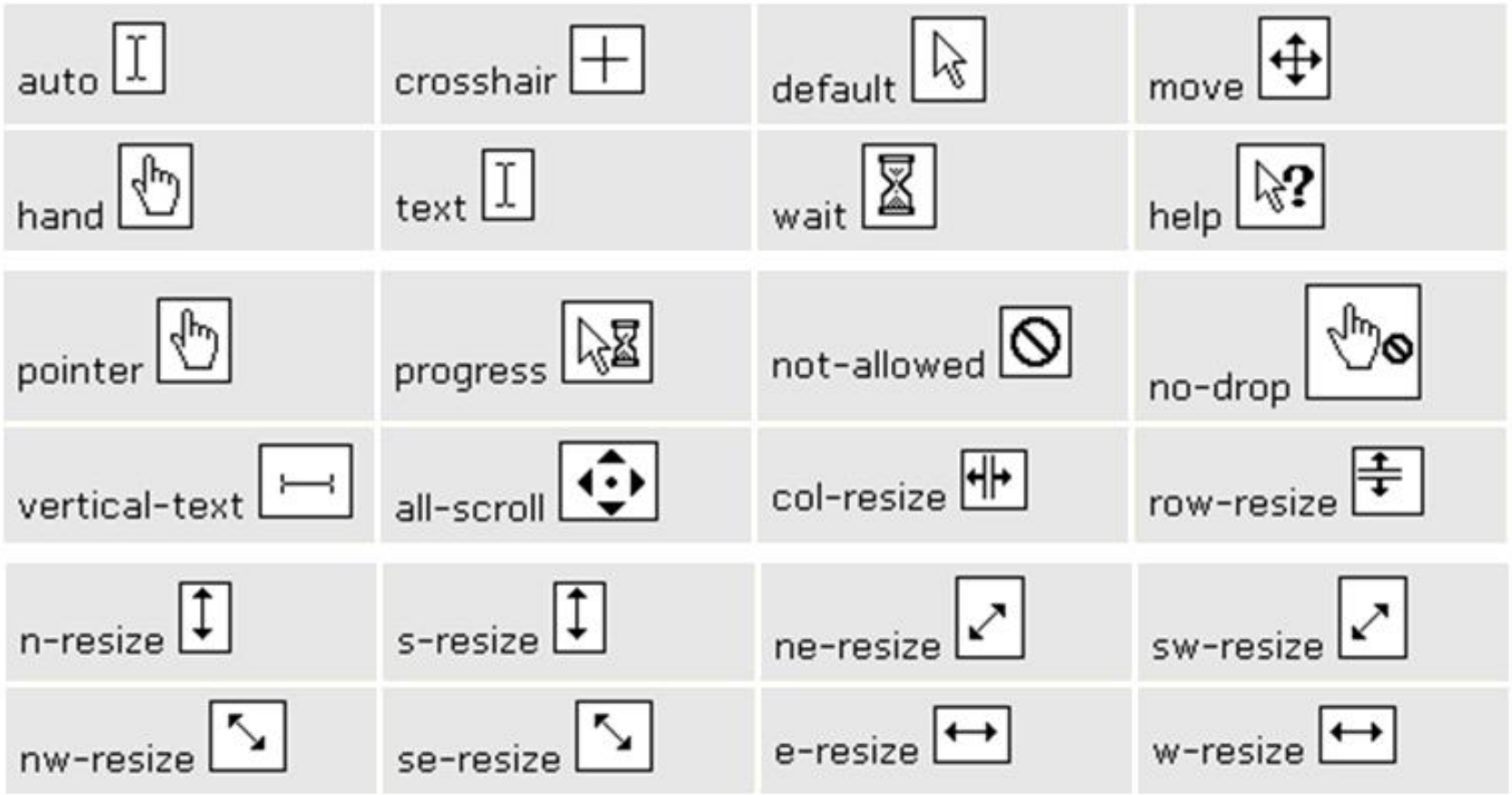


Different cursor types and their associated hot spot locations



Cursors and CSS

crosshair



This allows different visual feedback depending on the context.

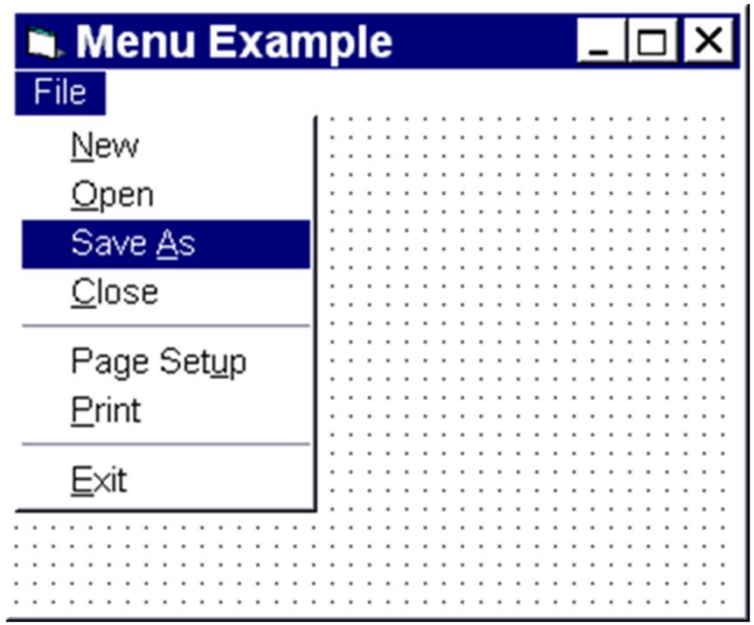
esempio

Menus

Choice of operations or services offered on the screen
Required option selected with pointer

Problem – take a lot of screen space

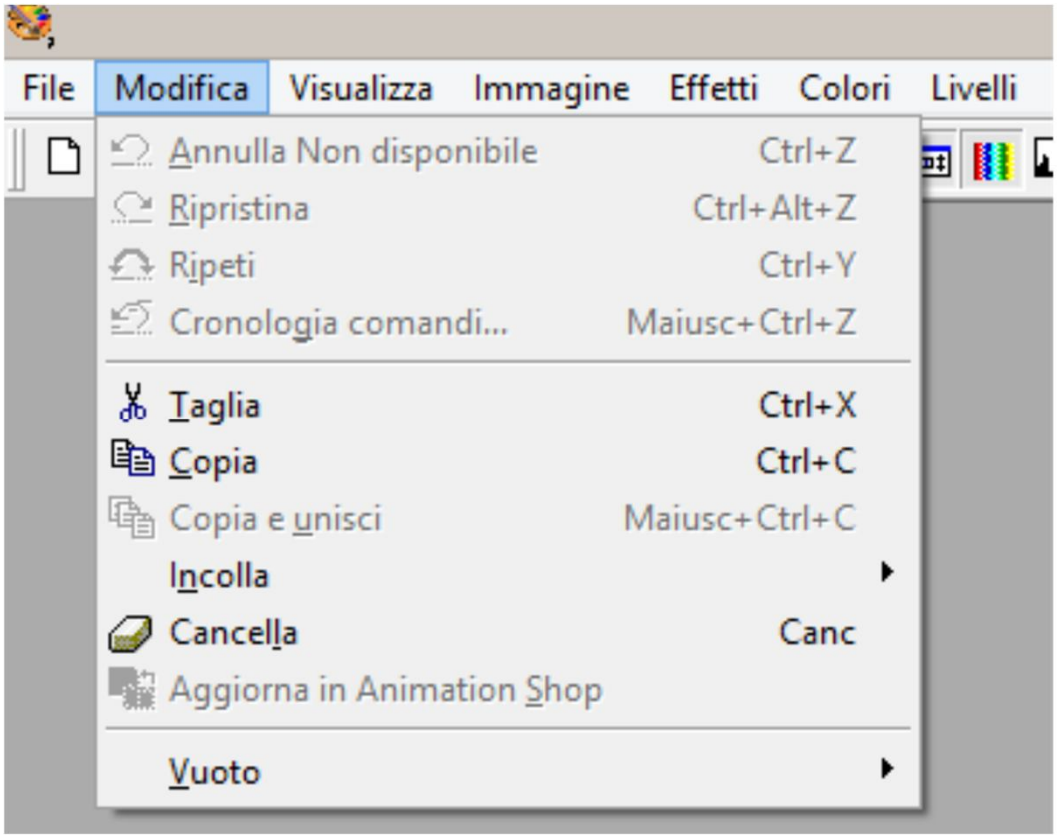
Solution – pop-up: menu appears when needed



Menus

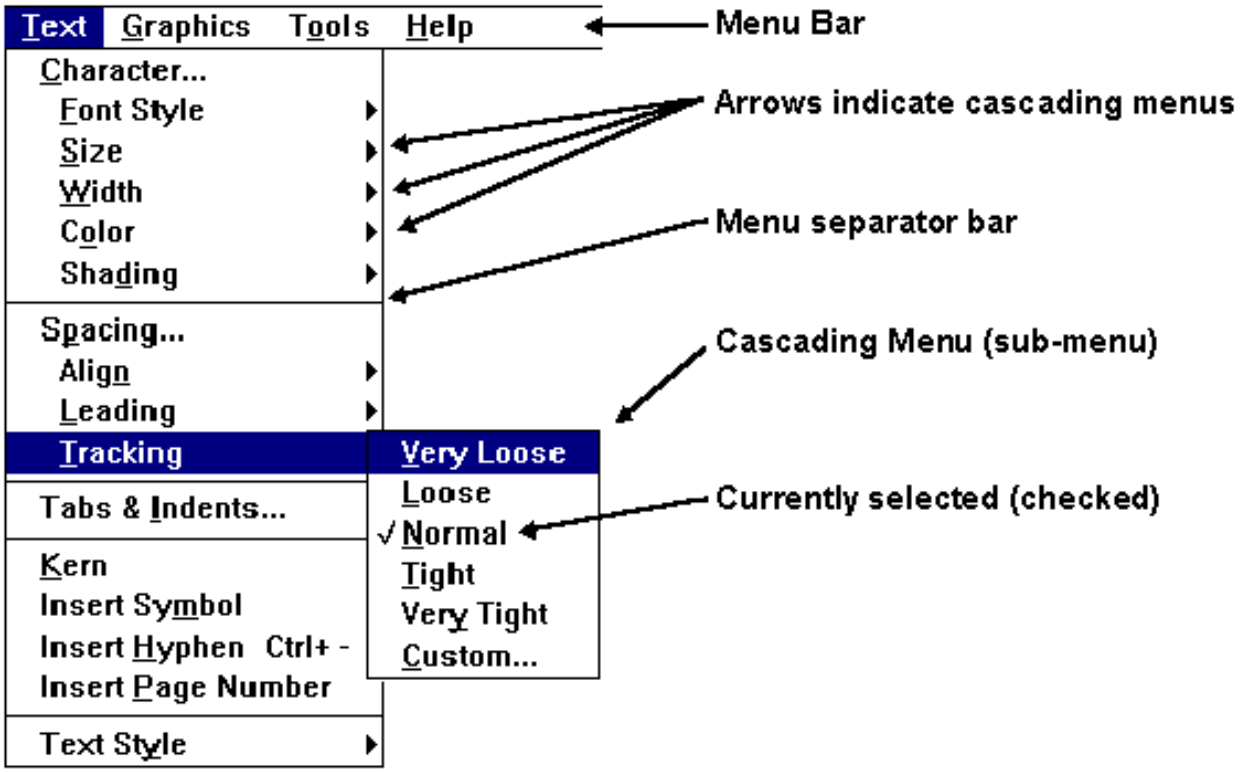
Conventions and guidelines:

- place **top-left** (more variety within web sites)
- **feedback** (color change),
- **greyed** items..
- **chunking**
  (with separators)



Menus

Menus are not effective when they contain too many items.. therefore… **sub-menus**



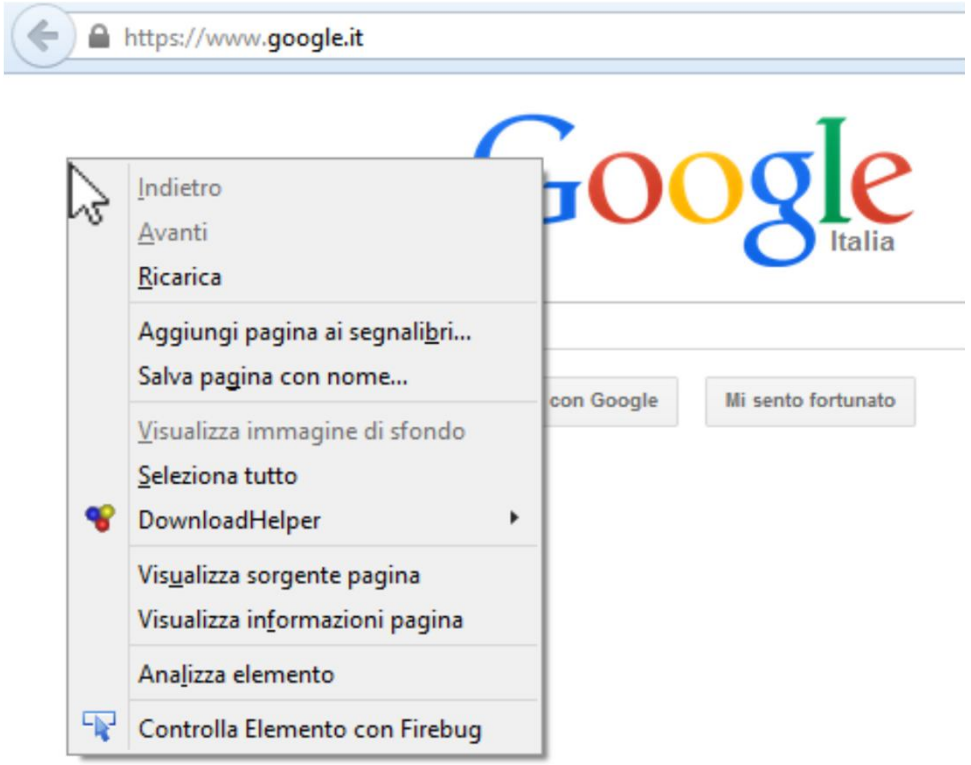
Kinds of Menus

Menu Bar at top of screen
(normally), menu drags down
pull-down menu - mouse hold and drag down menu
drop-down menu - mouse click reveals menu
fall-down menus - mouse just moves over bar!

Kinds of Menus

Contextual menu

- appears where you are
- **pop-up menus** with actions for selected object



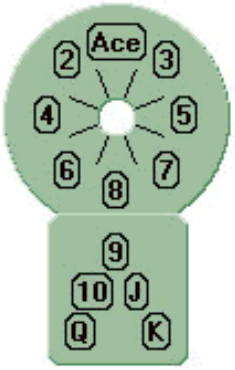
You can also create a **pop-up form** to display information to a user or to prompt a user for data.

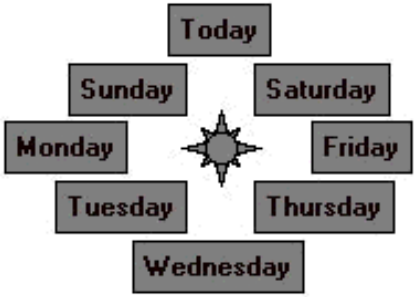
Pie-Menus

Arranged in a **circle**: the cursor starts out in the center of the pie.

Easier to select item (**larger target area**) therefore.. **low error rate**

Quicker (same distance to any option)
… but not widely used (space needed)





Example Generator

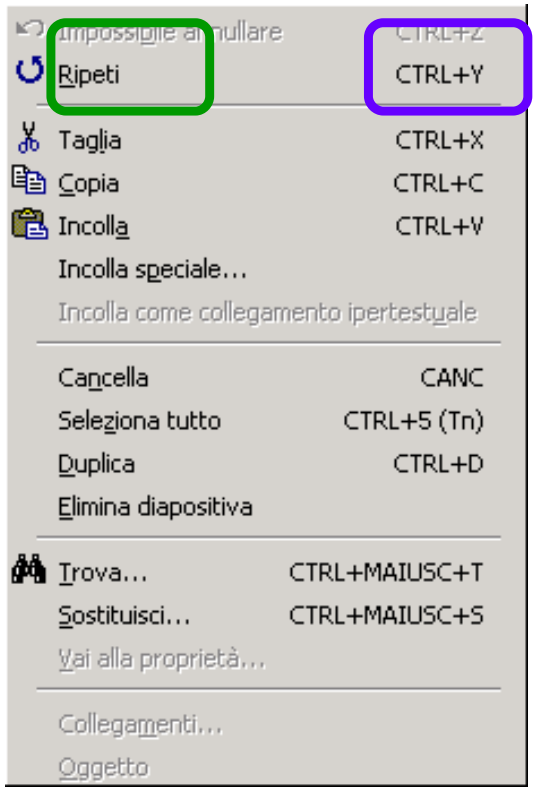
Keyboard accelerators

Short-cut keys can **reduce the number of interactions** for a given task

Two kinds:
- active when menu open
  usually first letter
- active when menu closed
  usually Ctrl + letter

Ideal maximum: 2 keys (+)

Not usable with cascading menus



Menus design issues

which **kind** to use

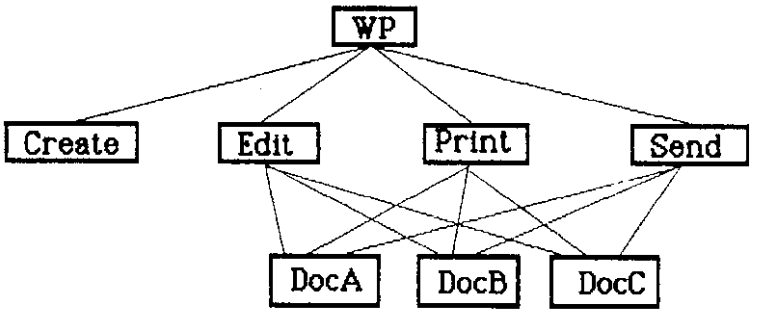
what to include in menus at all

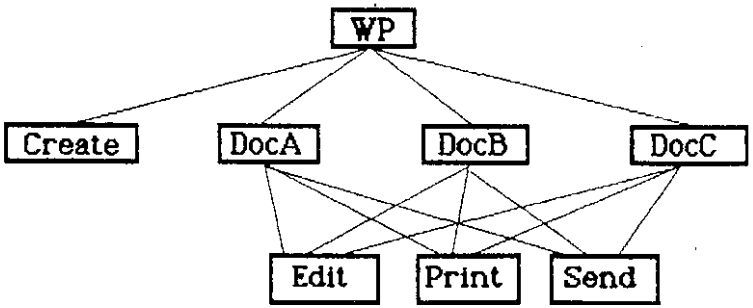
words to use (action or description)
(make item brief; begin with keyword; use consistent grammar, layout, terminology)

how to **group** items

choice of keyboard accelerators

order (frequency, sequence, separate opposite functions)



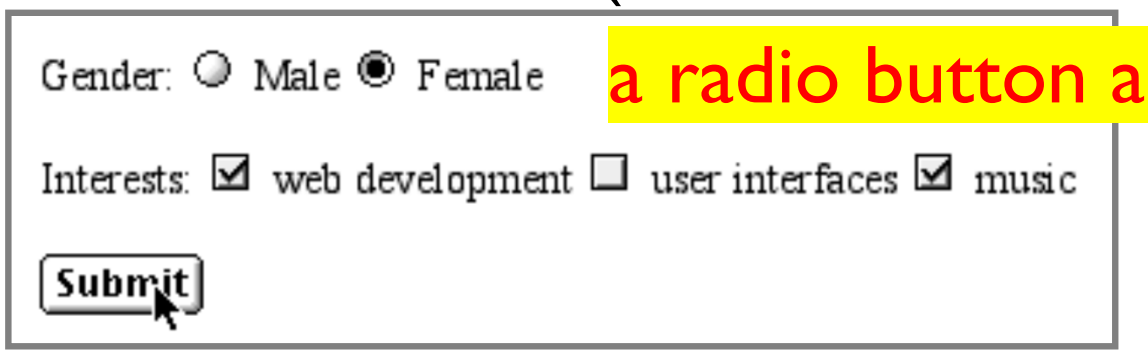


Buttons

Individual and isolated regions within a display that can be selected to invoke an action, to change the status or to select options within a form

They may provide a **feedback** on the **status** of the sytem

Special kinds
- **radio buttons** (set of mutually exclusive choices)
- **check boxes** (set of non-exclusive choices)



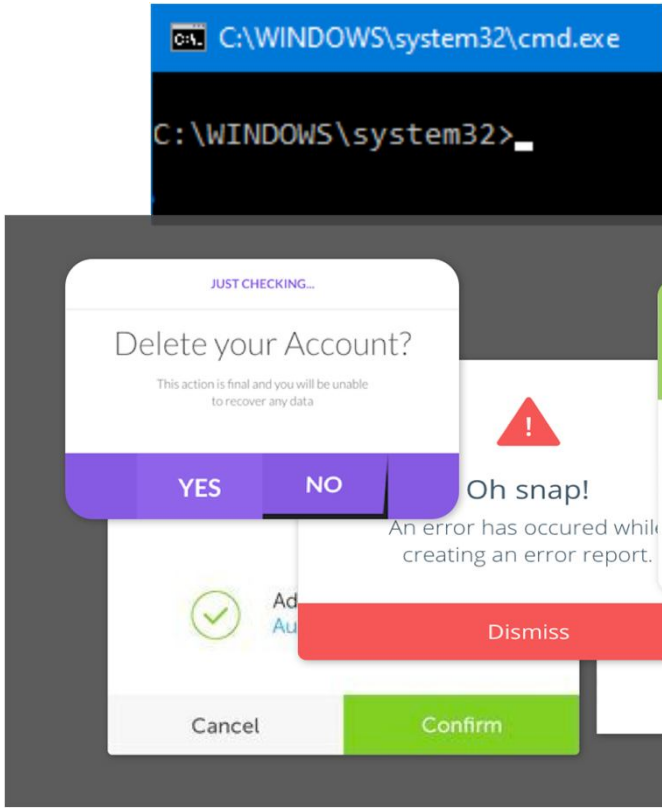
a radio button alone cannot be deselected

Who has the initiative?

Who has the initiative?
- **preemptive** interfaces: **computer**
- **non-preemptive** interfaces (e.g. WIMP): **user**

WIMP exceptions:
pre-emptive parts of the interface
(where the computer **steals**
the initiative to the user)

→ **Modal dialog boxes**:
come and won't go away!
Good for errors, essential steps
but use with care

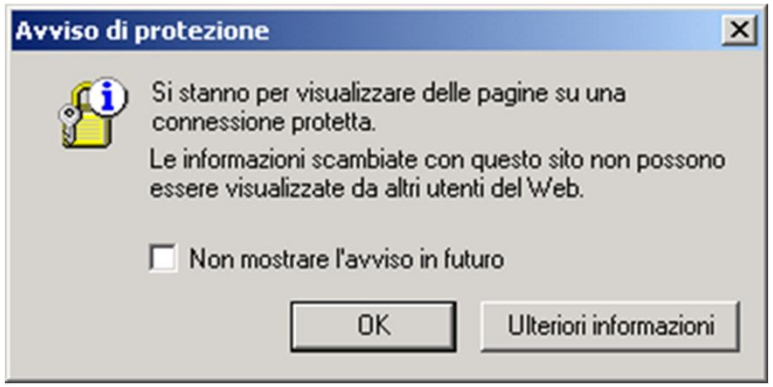


Dialogue boxes

Information **windows that pop up** to inform of an important **event or request information**..
and **once used.. disappear**

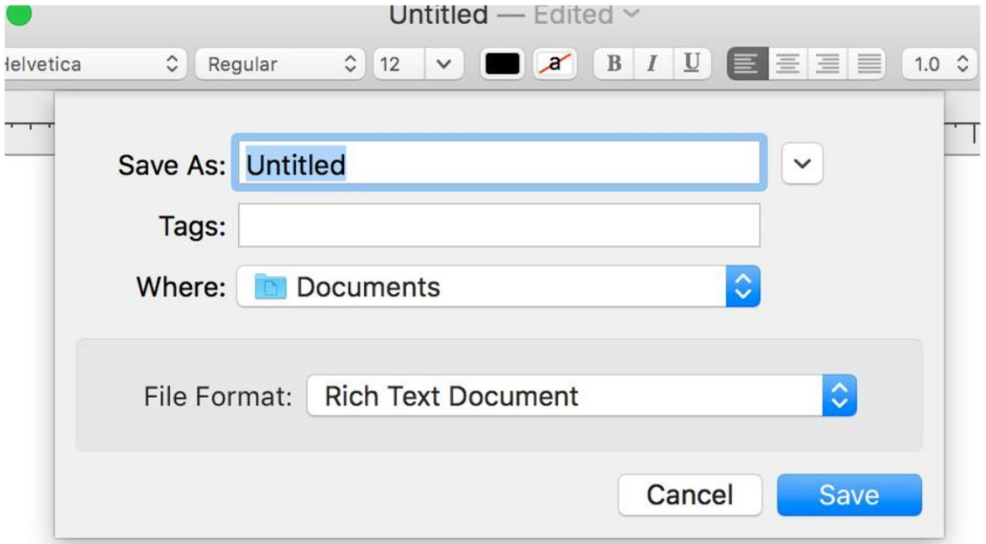
E.g: when saving a file, a dialogue box is displayed to allow the user to specify the filename and location. Once the file is saved, the box disappears.

Philosophy: divide complex tasks into simpler steps

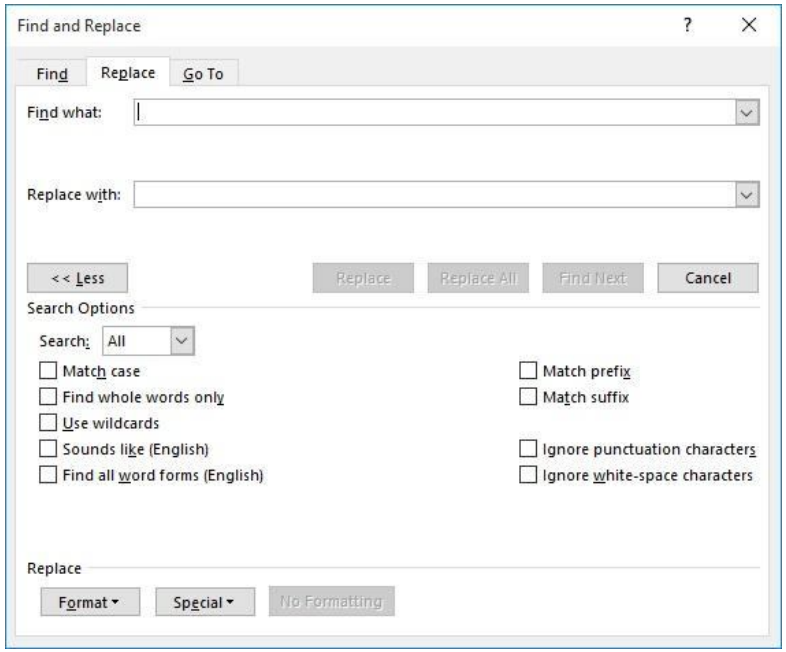


Modal and non modal dialogs

Modal → block interaction with the rest of the interface
Non-modal → allow interaction with other windows



Modal dialogs require **immediate attention**
Non-modal dialogs support **parallel activities**



Problems with Modal Interfaces

- Interrupt the user's **workflow**
- Force **immediate** decisions
- Increase **cognitive load**
- Can cause **errors** (e.g. accidental clicks)
- Reduce **flexibility** and control

When to Use Modal Dialogs

- When a **decision** is **required** before continuing
- For **critical actions** (e.g. delete, overwrite)
- To **prevent** serious errors
- When user **input is mandatory**
- When **focus** must be on a single task

Javascript Example

Dialogs are created with the dialog element.
showModal() opens a modal window.
show() opens a non-modal one.

Example

```
<script>
  const modal = document.getElementById("modalDialog");
  const nonModal = document.getElementById("nonModalDialog");

  function openModal() {
    modal.showModal();   // blocca interazione
  }

  function closeModal() {
    modal.close();
  }

  function openNonModal() {
    nonModal.show();     // non blocca
  }

  function closeNonModal() {
    nonModal.close();
  }
</script>
```